JAVA SWING · JDBC · MYSQL

AquaTrack

Use the station system.
Understand its code.

Installation, every operational screen, worked examples, five infographics, and the design behind the uploaded NetBeans project.

Project: Aquatrack3-gpt-claude_1
Offline learning edition • Searchable text • Linked source references
Verification limitations and the known test mismatch are included.

Contents

AquaTrack — user and developer handbook 3
Contents 3
Visual learning map 4
Installation and first login 5
First operating setup 8
Using every screen 8
Worked example 13
Daily checklist and backup 13
Troubleshooting 14
Developer guide 15
Rebuild the learning materials 16

How and why this AquaTrack project works 17
1. Model the business before the screens 17
2. Runtime architecture 17
3. Source map 18
4. Database relationships and constraints 20
5. Formulas and price selection 21
6. Order transaction: from click to committed ID 22
7. Payments and container movements 23
8. Accounts, passwords and session identity 23
9. Background work and navigation 24
10. Delivery, reports and printing 24
11. Extension map and limitations 25

Verification and import provenance 26
Import decisions 26
Current checks 26
Not exercised 27

AquaTrack — user and developer handbook

This guide documents **Aquatrack3-gpt-claude_1**, imported from the user's uploaded NetBeans project. It manages a water refilling station through Java Swing and MySQL: customers, pickup/delivery prices, sales, stock, riders, credit payments, returnable gallons, reports, and login accounts.

Download the illustrated PDF for offline reading. On GitHub, open the file and choose **Download raw file**. The PDF includes this README, the architecture chapter and verification notes, with embedded vector graphics and searchable text.

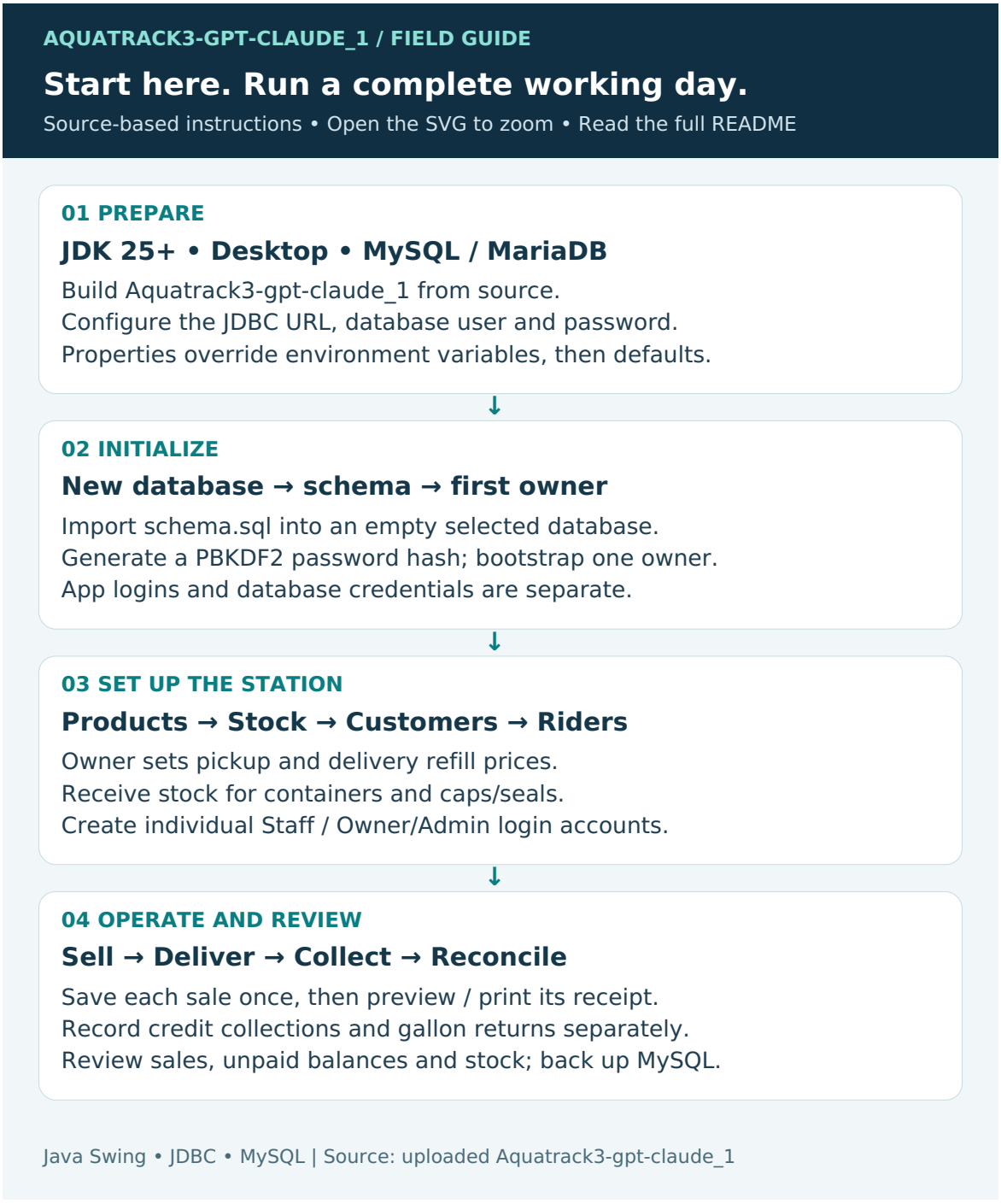
How and why the code works explains the implementation and its tradeoffs. **Verification and import record** distinguishes checks performed here from historical statements in the upload.

This is the `Aquatrack3-gpt-claude_1` project. The unrelated `GUI`, `LoginSystem`, and `LoginSystem_PT` folders are separate exercises. Their databases, credentials, and setup instructions do not apply here.

Contents

- Installation and first login
- First operating setup
- Using every screen
- Worked example
- Daily checklist and backup
- Troubleshooting
- Developer guide
- Rebuild the learning materials

Visual learning map



Graphic	Learn
Start and operate	Prepare the workstation, first login, records and daily tasks.
Sale and settlement	Pickup/delivery pricing, debt, collections and gallon counts.
Architecture	Screen → rules/models → DAO → JDBC → database.
Transactions and printing	Safe saves, current prices, payment acknowledgement and print snapshots.
Data model	Eight tables, historical amounts and customer-level balances.

Installation and first login

1. Requirements and source

Use a **JDK 25 or newer**, a desktop session, and a MySQL-compatible server. The project targets Java 25; a Java 21 runtime cannot load these compiled classes. The bundled Connector/J driver is under `lib/`. The supplied database DDL comes from MariaDB; use a current MariaDB server (10.4+ supports the relevant constraints). MySQL 8.0.16+ is an alternative to validate in your environment. Live database compatibility was not exercised during this import.

Windows users can run MySQL/MariaDB through XAMPP. Apache is needed for phpMyAdmin, but AquaTrack connects directly to the database and does not need a web server. The application does not create a database or default login on startup.

```
git clone https://github.com/ideki4444/JAVA_GIU_SWING_DATABASE_PT.git
cd JAVA_GIU_SWING_DATABASE_PT
```

Before this import is merged, select `codex/aquatrack3-current-handbook`. Then enter `Aquatrack3-gpt-claude_1` and check `java -version` and `javac -version`. Alternatively download the branch ZIP from GitHub and extract it.

The import includes Java source, tests, required libraries, shared NetBeans files, helper scripts, and clean setup SQL. Local database dumps, backup archives, compiled output, and `nbproject/private` settings are omitted. Copying the source does not copy the live database.

2. Build

From `Aquatrack3-gpt-claude_1`, in Windows PowerShell:

```
.\scripts\build.ps1
java -jar .\dist\Aquatrack3-gpt-claude.jar
```

Complete database setup before launching. To run the supplied automated suite use `.\scripts\build.ps1 -Test`; it currently stops at a known layout-test mismatch, documented in verification. The build without `-Test` compiles and packages the application. This distinction avoids claiming the uploaded test suite is entirely green.

In NetBeans, open the project folder, choose JDK 25+ in project properties, confirm the local Connector/J library, and use Clean and Build. Its main class is `aquatrack.Main`. NetBeans names its output **`dist/Aquatrack3-gpt-claude_1.jar`**, while the PowerShell script names it **`dist/Aquatrack3-gpt-claude.jar`**. Use the filename produced by your chosen build path. Keep the accompanying `dist/lib` folder with a distributed JAR.

Linux/macOS compilation, from the same project directory:

```
mkdir -p build/package-classes
find src -name '*.java' > sources.txt
javac --release 25 -encoding UTF-8 -cp 'lib/*' -d build/package-classes @sources.txt
java -cp 'build/package-classes:lib/*' aquatrack.Main
```

Windows classpaths use ; instead of :. Compilation requires no database connection. Swing windows require a graphical desktop.

3. Select the database connection

`Database.java` resolves settings in order: **JVM property → environment variable → default**. No source edit is required. An explicitly set empty value is still a configured value.

Setting	JVM property	Environment variable	Default
URL	`aquatrack.db.url`	`AQUATRACK_DB_URL`	`jdbc:mysql://localhost:3306/aquatrack?serverTimezone=Asia/Manila&connectTimeout=5000&socketTimeout=15000`
Database user	`aquatrack.db.user`	`AQUATRACK_DB_USER`	`root`
Database password	`aquatrack.db.password`	`AQUATRACK_DB_PASSWORD`	Empty string

PowerShell example, with credentials appropriate to your server:

```
$env:AQUATRACK_DB_URL = 'jdbc:mysql://127.0.0.1:3306/aquatrack?serverTimezone=Asia/
Manila&connectTimeout=5000&socketTimeout=15000'
$env:AQUATRACK_DB_USER = 'your_database_user'
$env:AQUATRACK_DB_PASSWORD = 'your_database_password'
java -jar .\dist\Aquatrack3-gpt-claude.jar
```

Use `export AQUATRACK_DB_USER='...'` and similar assignments in Bash. These affect processes launched from that shell; NetBeans needs matching settings in its launch environment or JVM options. Keep real passwords out of commits and shared command transcripts. `.env` files are not automatically loaded.

The database account is different from the AquaTrack login account. After schema creation, a dedicated database user generally needs SELECT/INSERT/UPDATE/DELETE on the application's database. Schema setup/migration needs additional DDL privileges. Desktop role controls do not replace control over database credentials.

4. Create a fresh database

The import supplies a **schema-only** version of the uploaded database definition. It contains eight tables and no station/customer data. Use a new, empty database; this script is not an upgrade script and intentionally fails if its tables already exist.

Start the MySQL client from the project directory:

```
mysql -u root -p
```

On Windows, if it is not on PATH:

```
& 'C:\xampp\mysql\bin\mysql.exe' -u root -p
```

Then execute:

```
CREATE DATABASE aquatrack CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE aquatrack;
SOURCE sql/schema.sql;
SHOW TABLES;
```

If `aquatrack` already exists, use a new name and update the JDBC URL accordingly. Do not drop an existing database to follow this tutorial. The expected tables are `users`, `customers`, `products`, `delivery_staff`, `orders`, `order_items`, `payments`, and `container_ledger`.

In phpMyAdmin, create/select the empty database first and import `schema.sql`. `SOURCE` is a MySQL command-line instruction, not a SQL statement for phpMyAdmin.

5. Bootstrap the first owner

There is no built-in login for a fresh installation. Generate a hash using the compiled project's helper:

```
java -cp build/package-classes aquatrack.util.PasswordUtil 'your-chosen-password'
```

Choose at least six characters; use a strong password for real use. NetBeans-only builds use `build/classes` instead. The command-line argument can remain in shell history. Copy the complete `pbkdf2_sha256$65536$...` result. In one MySQL session:

```
USE aquatrack;
SET @owner_username = 'station_owner';
SET @owner_full_name = 'Your Name';
SET @owner_password_hash = 'PASTE_COMPLETE_GENERATED_HASH';
SOURCE sql/bootstrap-owner.sql;
```

The result must say **Owner account created**. The script refuses when `users` already contains records or the values fail its basic checks. It checks the hash format, not whether you remember the password; use the actual helper output. Run this once during exclusive first-time setup. For phpMyAdmin, submit the SET statements and script contents together so variables share a session. Escape an apostrophe in SQL text by doubling it.

Launch the app, enter your chosen username and original plaintext password, and sign in. Create subsequent accounts through Accounts.

6. Existing installations and optional demonstrations

For an existing compatible pre-delivery-price database, back it up and close running clients before using `scripts/migrate-prices.ps1 -DatabaseName your_database`. It requires the PowerShell-built JAR, local MySQL tools, and their own authentication configuration. It does not automatically use the application's environment password. The script takes a backup, applies `sql/001_delivery_prices.sql`, and compares existing product/sales/payment/ledger records. Failure preserves the backup for inspection; it does not automatically restore over data.

The migration was recovered from the appended migration in the uploaded SQL. It adds nullable `delivery_price` when missing, fills only null refill rates from pickup prices (including archived refills), and adds a nonnegative check. Re-running preserves configured prices. Do not run the fresh schema against an existing installation.

For disposable training data after building on Windows:

```
.\scripts\create-demo.ps1 -DatabaseName aquatrack_demo_training01
.\scripts\run-demo.ps1 -DatabaseName aquatrack_demo_training01
```

The creation helper requires local MySQL tools, refuses an existing database, and prints the launch command. It uses the schema-only baseline and migration, then `scripts/DemoSeed.java`. Its intentionally synthetic accounts are `demo_owner` / `DemoOwner25!`, `demo_manager` / `DemoManager25!`, and `demo_staff` / `DemoStaff25!`. A username containing “manager” is not a third application role: the only roles are Staff and Owner/Admin. These fixtures are for isolated training, not live deployment. A database mentioned as already prepared in the uploaded notes does not exist automatically on your computer.

First operating setup

1. Sign in as Owner/Admin and add refill products with pickup and delivery prices in **Products & Prices**.
2. Add sellable Container and Cap/Seal products. Receive their opening quantities through **Inventory**.
3. Add customer names, addresses and mobile numbers. Even a walk-in sale needs a customer record.
4. Add riders through **Deliveries** if you offer delivery.
5. Create individual staff accounts through **Accounts**.
6. Practice the worked example below on an isolated database before real sales. Saved orders have no cancellation/refund workflow in this UI.

Using every screen

Navigation, sign-in and roles

The application opens a workspace containing sign-in and the operational screens. Enter both credentials. An unknown/inactive username reports **Account doesn't exist**, clearing both fields. A wrong password reports **Password is incorrect**, preserving the username and clearing the password. Successful authentication reports **Log-in Successfully**. These are the current source's behaviors, even where older uploaded notes describe generic rejection.

Screen queries and writes use background workers. While an operation is pending, controls and navigation are guarded; wait for completion instead of repeatedly clicking Save or exiting. Reopening a screen refreshes its records. Scroll the workspace/table when small windows or display scaling hide controls. Log out ends the session; unsaved drafts and the latest-receipt pointer are not durable across sessions.

Task	Staff	Owner/Admin
Dashboard, orders, deliveries, receipts	Yes	Yes
Add/edit customers	Yes	Yes
Record payments and gallon movements	Yes	Yes
Receive stock	Yes	Yes
Edit product catalog/prices and reorder levels	No	Yes
Archive/restore/delete customers	No	Yes
Reports and account management	No	Yes

Owner controls are partly UI-level. Account mutations additionally recheck the acting owner's current database record. This is still a desktop JDBC system, not a server authorization service.

Customers

Choose New, enter full name, address and contact number, then Save. Select a row to edit it. Search and active/archive filters help locate records. A mobile number must contain 11 digits starting with `09`; names and addresses are required and limited to 100 and 255 characters respectively.

The form checks duplicate names/contact numbers; the database also has a composite unique key on `(full_name, contact_no)`. Owners can Archive to hide a customer while preserving history, Restore to make them operational again, or Delete a customer with no dependent history. Settle outstanding obligations before archiving, or restore the customer to record further payments/returns. The Unpaid balances report includes archived debtors.

Products & Prices

Choose **New product**, name and category, then enter **Pickup / unit price**. For a Refill, also enter **Delivery refill price**. Enter **Low-stock alert at** for tracked products and Save. Values must fit the application's money/count validation; monetary inputs are nonnegative with at most two meaningful decimal places. Refill stock is untracked and its reorder threshold is zero.

Category	Price selection	Inventory
Refill	Pickup rate for Walk-in; delivery rate for Delivery	Not deducted; made on demand
Container	Single unit price	Sold quantity deducted on order save
Cap/Seal	Single unit price	Sold quantity deducted on order save

Search by name/category; searches are debounced and treat wildcard characters literally. **Current products**, **Archived**, and **All** control visibility. Status distinguishes Available, In Stock, Low Stock, Out of Stock and Archived. Select a product to edit. Filtering can hide the selected row without discarding the edit; check the form's record identity before saving.

Archive stops new use without removing history; Restore reverses that. Delete is restricted for products already used by orders. Product names are unique in the database. Create a distinct product rather than casually changing an established product's business meaning/category. A refill does not automatically consume a cap or sellable container; include those as sale lines if sold.

Inventory

Search active stock-tracked products, select an item, enter a positive whole number in Stock received, and choose Add to stock. This increments the existing quantity. Owners can change the low-stock alert threshold. Stock at or below that threshold is low; zero is out of stock. Refills do not appear here. There is no general stock-writeoff/adjustment-history module; ordinary deductions occur when sales are saved, including deliveries still pending.

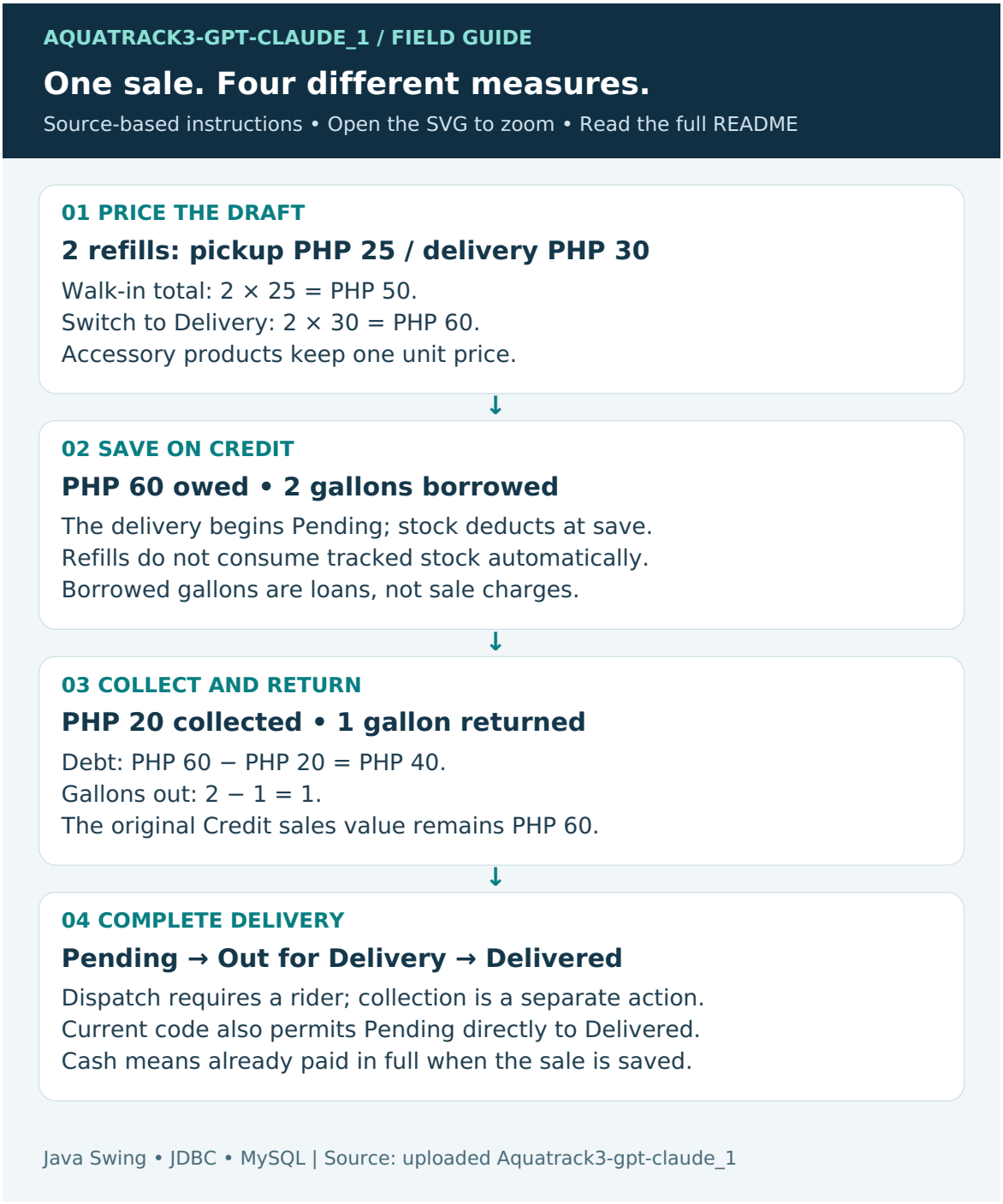
New Order

1. Choose an active customer and **Walk-in (served now)** or **Delivery (rider assigned later)**.
2. Choose **Cash, paid now** only when fully paid. Choose **Credit, add to balance** for money still owed, including cash-on-delivery before collection.
3. Enter gallons borrowed/returned (0–500 in each spinner), or leave zero.
4. Select a product, enter Qty (1–999 per addition), and choose Add to order. Adding the same product combines its quantity.
5. Review every line and total. Remove item removes the selected draft line. Clear order resets the draft. Refresh choices reloads current customer/product choices.
6. Changing Walk-in/Delivery reprices refill lines. Review the updated total; accessories keep their single price.
7. Choose **Save and preview receipt** once. Wait for the result, then print or close the preview.

The database rechecks customer activity, product activity, prices and stock. If another workstation changed a price, the transaction rolls back and the draft receives current prices for another deliberate Save. Insufficient stock or excessive returns also prevent the save. A return may not exceed existing gallons out plus gallons borrowed in the same sale.

Walk-in orders start Delivered; delivery orders start Pending. Order header, item rows, tracked stock deductions and order-linked gallon movements commit together. Gallon loans are not a charge and do not change sellable-container stock.

After a successful save, the draft is cleared and the order ID is retained for receipt access. **View last receipt** becomes the saved-order reference; a receipt-loading failure can be retried by ID. Do not re-enter the sale because printing failed. The app has no general historical-receipt search, saved-order edit, cancellation, refund, discount, tax, cash tender/change or split-payment field. For a part-payment, create the full Credit sale and record the amount received separately.



Deliveries

Save a Delivery order first. In Deliveries, select it, add/select a rider and choose Assign rider. Choose Out for delivery at dispatch, then Mark delivered on completion. The open-only filter excludes Delivered orders; disable it to include completed deliveries.

Dispatch requires Pending status and a rider. Delivered orders cannot change riders. The implementation also permits **Pending directly to Delivered without a rider**; the full dispatch sequence is an operating recommendation, not a strict invariant. A stale status update is rejected and refreshed. Delivery completion neither collects payment nor changes Credit to Cash. Rider records are separate from user logins; the UI does not provide full rider editing/deactivation management.

Containers & Balances

The default **Only customers with gallons out or a balance** filter hides settled customers. Uncheck it to find a new customer's first standalone loan. Archived customers remain excluded; restore them through Customers if needed.

Select a customer and enter **Payment received (PHP)**, then Record payment. The amount must be positive and no larger than the current unpaid balance. The DAO locks the customer, recalculates the balance, inserts the payment and commits before returning an acknowledgement. The confirmation gives payment reference, amount, customer, previous and remaining balance; zero is Fully paid. If a subsequent refresh fails, the app distinguishes that from a failed payment. Keep the reference and refresh rather than recording the same collection twice.

For a movement outside a sale, enter the gallon count and choose Record gallons returned or Record gallons borrowed. Counts must be positive; returns cannot exceed the current outstanding count. These operations also check/write under a customer lock. Payments apply to the customer's combined debt, not to a selected invoice.

```
Unpaid = total of Credit orders − recorded payments
Gallons out = all Borrowed quantities − all Returned quantities
```

The screen totals cover active customers; the Dashboard's ledger total and archived-inclusive debt report can have a broader scope. There is no payment/ledger reversal or full movement-history editor in this UI.

Dashboard and Reports

Dashboard shows today's order value, open deliveries, gallons out and low-stock products, plus the seven-day sales chart and recent orders. Sales include Cash, Credit and pending deliveries on the date created. They are not cash collected or profit. The chart initially occupies 45% of the lower split; manual divider adjustments are retained during resizing.

Owners can choose Daily sales, Monthly sales, Best customers or Unpaid balances in Reports. Daily uses year/month and omits days without sales; Monthly uses year and fills zero-sales months; Best customers ranks all-time order value; Unpaid balances shows current debt, including archived customers. Date controls do not filter the latter two. Print is available for the successfully loaded snapshot. Changing selection while loading supersedes an older result.

A later credit payment reduces debt but does not convert the original order to Cash or add a second sale. Reconcile actual cash collections separately; there is no full cash-drawer closeout or profit module.

Receipt and report previews

Sales receipts use an **80 mm roll** with length fitted to the sale, wrapped details and black monospaced text. Preview offers Zoom, minus/plus, Print and Close. It deliberately lacks paper orientation and Previous/Next controls. Select a matching thermal-printer driver and adequate form length; actual feed/cutting behavior depends on hardware and has not been verified here.

Reports use **A4** with portrait/landscape selection and page navigation. Preview and print use the same document snapshot so an asynchronous screen refresh does not silently replace the printed data. Print opens the OS print dialog. A PDF printer is an OS feature; the application has no separate built-in PDF-export workflow. Closing the preview or canceling printing does not undo or repeat a sale.

Saved prices and amounts remain historical. Customer/product descriptions are read from current database records when reloading a receipt, so later edits can change its wording. Keep an external printed/PDF record when an exact historical presentation is needed.

Accounts

Owner/Admin opens Accounts. Choose New account, enter a unique username (3–50 letters, digits or underscores), full name (up to 100 characters), Staff or Owner/Admin role, initial password (at least six characters), and Account can sign in, then Create account.

Selecting a row fills the editing form. Save changes updates username/name/role/activity **without changing the password**. Reset password is a separate action/dialog. Save/Discard/Cancel protects unfinished edits when leaving or changing selection. Editing your own details refreshes the session's displayed identity.

Account writes lock the user rows in ID order and recheck the actor's active owner status. You cannot deactivate or demote your own signed-in account, and at least one active owner must remain. Username and normalized full-name duplicates are checked. There is no public registration/email recovery. Disabling a different user blocks later sign-in; not every business operation continuously revalidates all existing sessions.

Worked example

Use a fresh training customer, no existing debt or gallons out. Create a refill at PHP 25 pickup and PHP 30 delivery.

1. Add two refills to a Walk-in draft: total PHP 50.
2. Switch to Delivery: total becomes PHP 60. Choose Credit, borrow two gallons, return zero, and save.
3. Assign a rider, dispatch and complete delivery. Debt remains PHP 60 until collection is entered.
4. Record PHP 20 received and one gallon returned. Debt becomes PHP 40; gallons out becomes one.
5. Reports retain PHP 60 of original Credit sales. Collection does not create another sale.
6. If selling caps or a container too, add explicit product lines: those affect stock and sale value independently of loan counts.

Daily checklist and backup

When	Check
Opening	Database running; correct workstation configuration; individual login; prices and low stock reviewed.

When	Check
Each sale	Correct customer, fulfillment, payment type, prices, quantities and loans; save once; keep the order reference.
Dispatch	Rider assigned, progress updated; collections recorded separately.
Closing	Review pending deliveries, sales, unpaid balances, stock and gallons; reconcile cash separately; back up and log out.

The live data is in MySQL, not GitHub. Stop data entry during backup/restore. Use an authorized database account:

```
mysqldump -u root -p --single-transaction --databases aquatrack --result-file=aquatrack-backup.sql
```

`mariadb-dump` may replace `mysqldump`. Store backups privately outside Git. Test restoration on an isolated server before replacing live data. In a MySQL client, `SOURCE /path/to/aquatrack-backup.sql;` loads a dump; use forward slashes on Windows. A dump can replace tables, so preserve the destination data first. Back up the application configuration and matching program version as well.

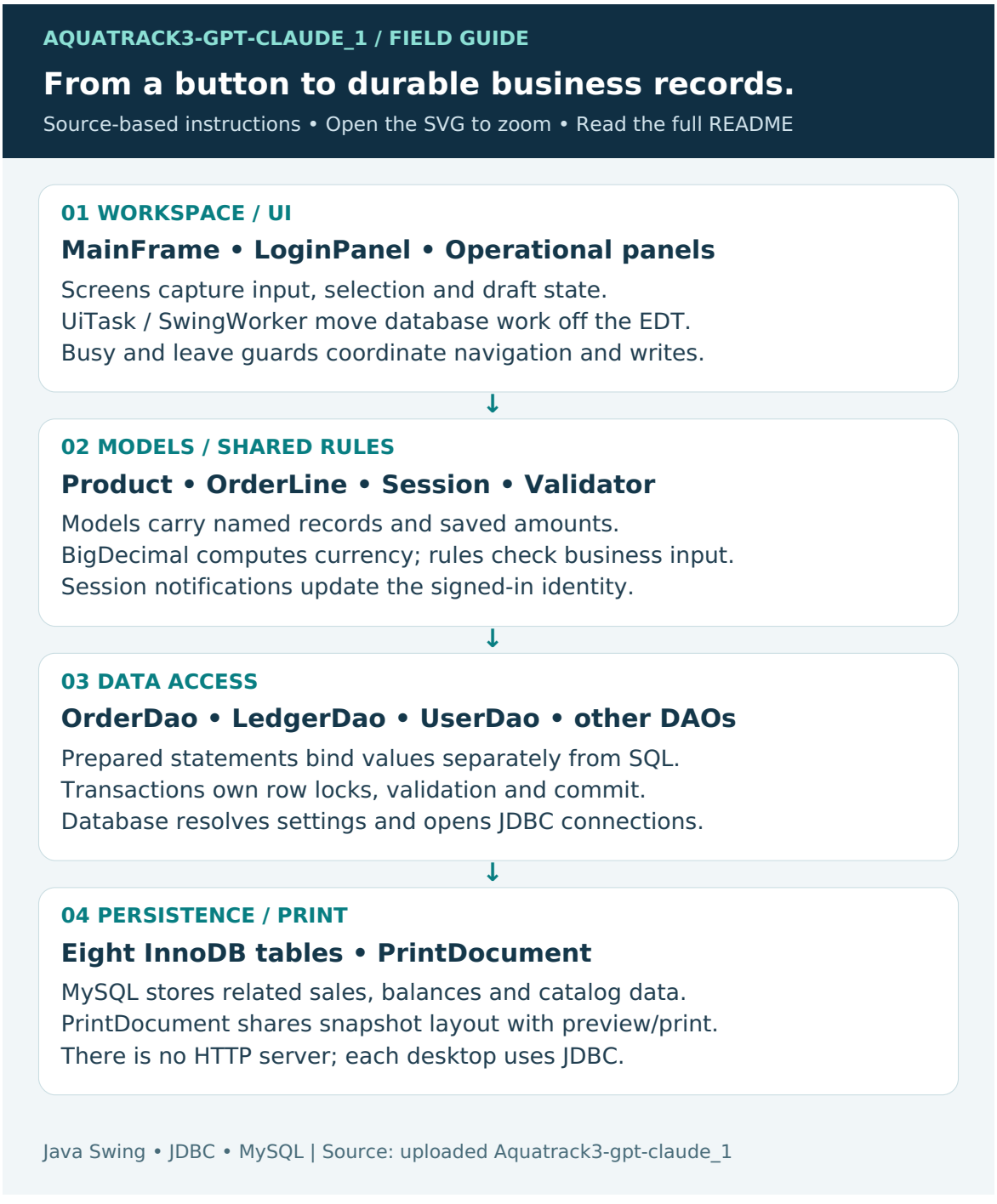
Troubleshooting

Symptom	Action
`javac` absent / release 25 unsupported	Install a full JDK 25+, update PATH, reopen the terminal.
Unsupported class version	Launch with Java 25+ matching the source target.
`build.ps1 -Test` stops on width 400 versus 420	Known uploaded test/source mismatch; see verification. Plain build packages without running tests.
Driver missing	Keep Connector/J on the classpath or the packaged JAR's `dist/lib` beside it.
Connection failure / access denied	Check DB service, host/port, effective property/environment values and database grants. App credentials are separate.
Table/`delivery_price` absent	Use the correct database; import clean schema only into an empty DB, or apply the compatible migration to an older schema after backup.
No account found	Bootstrap a fresh installation or check active status; enter plaintext password, not its stored hash.
Product not available	Restore archived items; refresh choices; receive tracked stock before selling.
Customer not in balance list	Disable open-items-only filter; restore an archived customer first.
Price changed	Review newly priced draft and save deliberately again. The previous attempt rolled back.
Saved payment/order but refresh/ print failed	Use saved reference and refresh/retry preview; avoid duplicate entry.
Controls seem inaccessible	Scroll workspace/table; wait for pending operations; check display scaling.

Symptom	Action
Receipt clipped by printer	Use matching 80 mm driver/form length and test hardware; report orientation controls apply to reports only.

For lost-owner recovery, a database administrator can generate a new PasswordUtil hash and update the exact intended user's hash/activity after verifying their identity and affected row. The first-owner script deliberately does not overwrite an existing account.

Developer guide



Read the architecture chapter for class responsibilities, data relationships, formulas, row-lock transactions, session/UI coordination, printing and extension points. “Why” explains engineering benefits inferred from the implementation, not undocumented intentions of the original author.

The supplied usability protocol and blank observations CSV are preserved as historical reference notes. Referenced survey documents, private backups and previously prepared databases are not included; the notes do not establish measured user-study results.

No runtime Java files were changed for this import. Clean SQL and documentation were added; historical claims in the uploaded README are not treated as proof of a current passing test run. See verification.

Rebuild the learning materials

The SVG generator uses Python's standard library. The PDF exporter requires Python 3.10+, packages in requirements.txt, and WeasyPrint's native Pango/Fontconfig libraries. See WeasyPrint installation for your OS. These are documentation tools, not Java runtime dependencies.

From the repository root on Linux/macOS:

```
python3 docs/tools/build_infographics.py
python3 -m venv .venv-docs
.venv-docs/bin/python -m pip install -r docs/tools/requirements.txt
.venv-docs/bin/python docs/tools/build_handbook.py
```

On Windows use `py -3` to create the environment and `.venv-docs\Scripts\python.exe` for the Python commands after installing native libraries. Edit the Markdown and diagram generator, rebuild, then commit the text, SVGs and PDF together. Inspect dense tables, all figures and links before publishing.

How and why this AquaTrack project works

This chapter describes the uploaded **Aquatrack3-gpt-claude_1** code. Read the README for installation and operation. Statements of behavior come from source; explanations of benefits are engineering interpretations. Proposed extensions are not existing features.

1. Model the business before the screens

A sale, payment, loaned gallon and delivery milestone can happen on different days. The program records them separately. Orders answer what was sold; payments reduce customer debt; container movements track what customers must return; delivery status records fulfillment. Selling a new container is distinct from lending a returnable gallon.

This separation explains why marking Delivered does not collect money, why a Credit sale remains Credit after collection, and why a returned gallon does not increase sellable product stock. It also means there is no invoice-specific payment allocation: payments belong to a customer.

2. Runtime architecture

`Main` starts Swing. `MainFrame` coordinates the workspace, login, navigation and session-ending actions. `LoginFrame` is a compatibility wrapper and `LoginPanel` handles credential input/authentication. Screen actions invoke DAOs, which open JDBC connections through `Database`. There is no HTTP backend, ORM or separate application server.

Package	Responsibility	Reason and tradeoff
`ui`	Widgets, action listeners, state, feedback, background tasks and previews	Tasks stay close to their controls, but panels contain application coordination as well as presentation.
`model`	Typed users, products, customers, orders and calculated summaries	Named values are clearer than arbitrary result-array positions. They are ordinary Java objects, not automatically persisted entities.
`dao`	Parameterized SQL, mapping, transaction boundaries	Keeps queries out of forms and allows connection test doubles. Different operations still have different validation guarantees.
`db`	Connection settings and small JDBC helpers	Centralizes credentials and resource handling without a connection pool.
`util`	Validation, money rules, delivery rules, hashing, sessions, ranking and print documents	Shared rules reduce duplication and can be tested independently of a real database.

3. Source map

All links below point to the imported source tree. Read the class with its corresponding screen rather than assuming a class name represents an entire service layer.

Class	Responsibility
Main	Launches the Swing application.
CustomerDao	Search, edit, archive and history-aware customer deletion.
DeliveryStaffDao	Rider records and active-rider lookup.
LedgerDao	Derived balances, locked payments and standalone container movements; committed PaymentResult.
OrderDao	Atomic sale creation, receipt lookup, rider assignment and conditional delivery updates.
ProductDao	Catalog/search, pickup and delivery prices, stock increments and thresholds.
ReportDao	Read-only sales aggregates and dashboard data.
UserDao	Account profile and password mutations with owner authorization and ordered locks.
Database	Resolves JVM/environment/default settings and creates JDBC connections.
Customer	Customer identity/contact/archive state.
CustomerAccount	Calculated gallons out and unpaid balance.
CustomerOrder	Joined order header, lines, rider and gallon details.
CustomerTotal	Accumulator for customer ranking.
DeliveryStaff	Rider identity independent of login accounts.
OrderLine	Quantity, price and line total snapshot.
Product	Category, pickup/delivery rates, stock and availability predicates.
User	Login identity, role, activity and password hash.
AccountsPanel	Create/edit accounts, dirty-form protection and independent password reset.
ContainersPanel	Payments, acknowledgement and borrowed/returned gallon entry.
CustomersPanel	Customer forms, searching and archive actions.
DashboardPanel	Metrics, recent orders and seven-day chart.
DeliveriesPanel	Riders, dispatch and completion controls.
InventoryPanel	Search, receive stock and update low-stock thresholds.
LoginAuthenticator	Testable credential outcome logic.
LoginFrame	Compatibility window around login UI.
LoginPanel	Credential UI and asynchronous authentication.
MainFrame	Workspace navigation, session display and pending-operation guards.

Class	Responsibility
NavIcon	Navigation icon drawing.
OrderPanel	Draft cart, repricing, save and receipt retry.
PrintPreviewDialog	Document rendering, zoom, page format and printing.
PrintPreviewToolbar	Shared preview controls.
ProductsPanel	Search and editing for catalog, category and both refill rates.
ReceiptDialog	Receipt-specific preview entry point.
ReportsPanel	Background reports, date selection and captured print content.
UiKit	Shared theme, layouts, tables, widgets and feedback.
UiTask	Background execution and EDT completion wrapper.
WaterArtwork	Java2D sign-in illustration.
WorkspacePanel	Scrollable host for operational content.
DeliveryRules	Allowed delivery actions and user-facing rejection messages.
Fmt	Currency and date formatting.
OrderRules	Stock/gallon checks and total addition.
PasswordUtil	Salted PBKDF2 hash generation/verification and setup CLI.
PrintDocument	Printable document model, pagination, layout and rendering shared with preview.
Ranking	Customer grouping and descending spending ranking.
Receipt	Formats receipt content from a saved order.
Session	Current user and EDT-dispatched identity-change notifications.
Validator	Required text, money, mobile and integer checks.

4. Database relationships and constraints

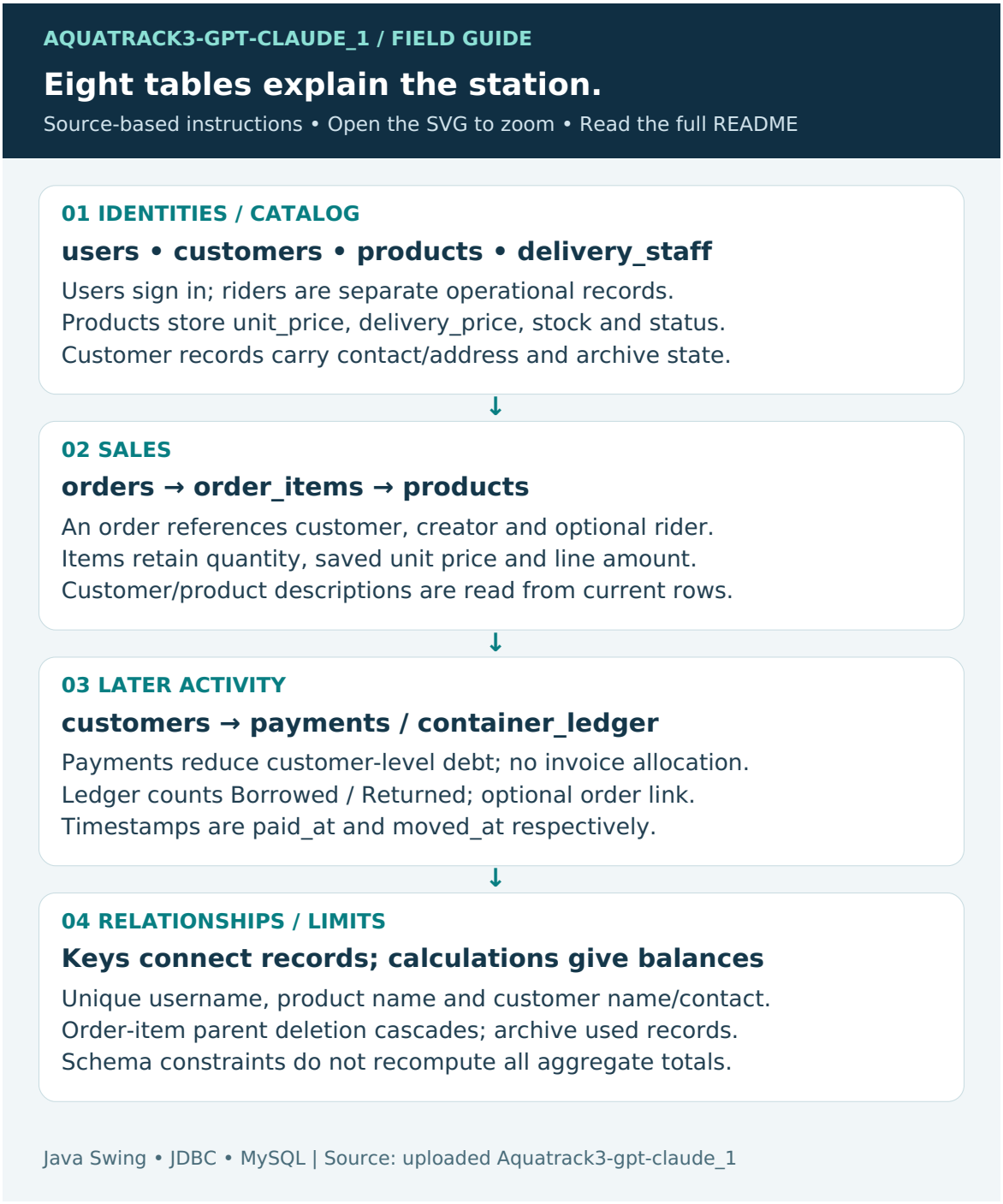


Table	Stored facts and relationships
`users`	Primary key user_id; unique username; PBKDF2 hash; full name; Owner/Admin or Staff; active flag. Orders reference their creator.
`customers`	customer_id; name/address/contact; archived flag; created_at; composite unique name/contact key. Parent of orders, payments and movements.
`products`	product_id; unique product name; category; decimal unit_price and nullable delivery_price; stock, threshold and activity. Referenced by order items.
`delivery_staff`	staff_id; rider name/contact/activity. Orders may have no rider initially.

Table	Stored facts and relationships
`orders`	order_id; order_date; customer; order/payment type; decimal total; status; optional rider; creator user.
`order_items`	item_id; order/product keys; quantity; saved unit price and line total. Header-to-items FK uses ON DELETE CASCADE.
`payments`	payment_id; customer; positive amount; paid_at. No order_id or collecting-user column.
`container_ledger`	ledger_id; customer; Borrowed/Returned; positive quantity; moved_at; optional order_id. Standalone movements have no order link.

The supplied SQL uses InnoDB and utf8mb4. Prices and monetary totals are DECIMAL(10,2); the application cannot store arbitrarily large order totals even if Java can calculate them. Quantity/amount checks and foreign keys protect the declared relationships, but the database does not independently recompute every order total from line items.

Normalization avoids copying customer details into each sale. Historical unit prices and totals are intentional snapshots; customer names/address and product names remain live references. That is why old receipts retain monetary values but may show edited descriptions.

The composite customer unique key is not equivalent to separate uniqueness of name and phone; the UI performs additional duplicate checks. Product names and usernames have individual unique constraints. Account full-name uniqueness is enforced by account mutation logic.

The clean schema was extracted from this upload's DDL, without INSERT data, database selection or auto-increment history. It is a fresh-install script, not an automatic migration framework. A schema-only pre-delivery-price baseline supports the supplied demo helper; the appended migration was extracted into its expected filename.

5. Formulas and price selection

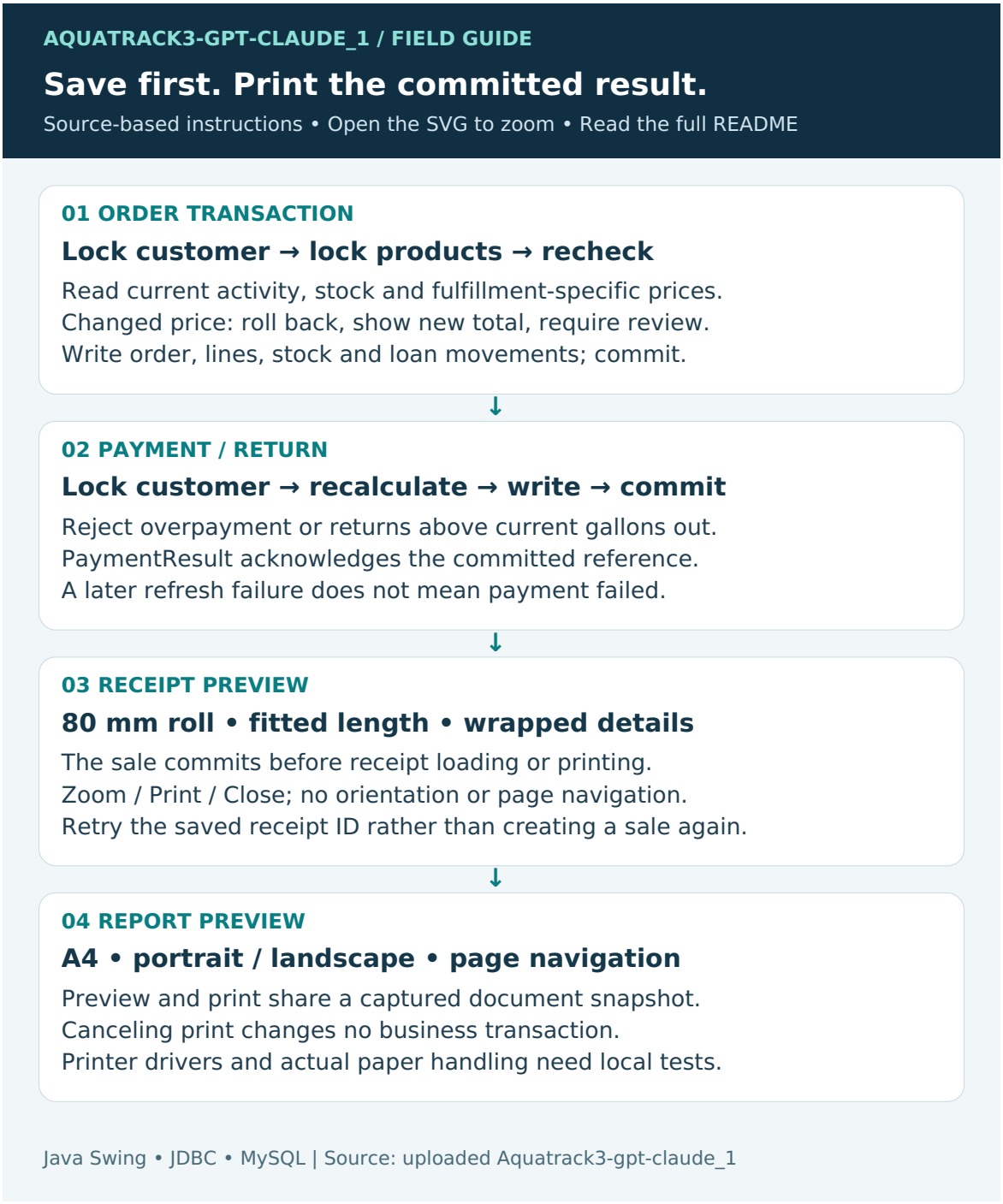
```
Refill rate = pickup unit_price for Walk-in
            = delivery_price for Delivery (fallback to pickup if null)
Accessory rate = unit_price for either order type
Line total = quantity × selected rate
Sale total = sum(line totals)
Unpaid balance = sum(Credit order totals) − sum(customer payments)
Gallons out = sum(Borrowed quantities) − sum(Returned quantities)
Stock after sale = current stock − quantity sold (tracked categories only)
Low stock = active AND tracked AND stock <= threshold
```

`Product.priceForOrderType` centralizes fulfillment-dependent price choice. `OrderLine.priced` and `OrderRules.total` use BigDecimal, avoiding binary floating-point error in financial arithmetic. SQL stores matching decimals. Chart geometry may use floating point without changing the saved accounting values.

For two refills at PHP 25 pickup/PHP 30 delivery, switching a draft from Walk-in to Delivery changes PHP 50 to PHP 60. A PHP 20 later payment leaves PHP 40 debt while original Credit sales remain PHP 60. Borrow two/return one leaves one gallon out. These are independent measures.

Sales reporting groups orders by their creation date; it includes uncollected Credit and pending deliveries. Database timestamps and Java local dates both participate in reporting, so keep workstation/server clocks and the configured Asia/Manila timezone consistent.

6. Order transaction: from click to committed ID



1. `OrderPanel` validates customer/cart/spinner input and captures the requested sale. UI stock checks provide quick feedback, not final authority.
2. `OrderDao.placeOrder` opens one connection, disables autocommit and locks the active customer with SELECT FOR UPDATE.

3. Each product is locked and reread. The DAO checks current activity, positive quantity and available stock, selects the current fulfillment-specific rate and deducts tracked stock on the transaction connection.
4. If any price differs from the reviewed draft, PricesChangedException carries updated lines. The transaction rolls back its writes. The UI shows revised values and requires a deliberate second Save.
5. The DAO checks returned gallons against previous outstanding plus newly borrowed. It inserts the header, item lines and nonzero order-related movements, then commits once and returns the generated order ID.
6. The UI clears the committed draft and retains that ID before receipt lookup. Loading/printing is subsequent work. A preview failure cannot undo the sale; retrying its ID avoids an accidental second order.

Why lock rows? Two clients may both see old stock. The second transaction must wait and then check the remaining quantity rather than selling the same units. The same customer lock coordinates order-related gallon checks with standalone movements and payments.

Locks are not a complete distributed retry protocol. Overlapping carts can acquire product locks in different orders and deadlock. A lost connection around commit can leave the client uncertain whether a sale committed. There is no general idempotency key or automatic refund flow; inspect persisted records before manually repeating an ambiguous save.

7. Payments and container movements

`LedgerDao.recordPayment` validates positive decimal money, begins a transaction, locks the active customer and recalculates their debt using that same connection. It rejects overpayment, inserts the payment, obtains a reference and database timestamp, constructs an immutable PaymentResult, and returns it only after commit.

`recordMovement` validates Borrowed/Returned and positive quantity, takes the same customer lock and checks the latest balance before inserting. Returns cannot exceed the count outstanding. These are transactional read/check/write operations in this project; they must not be described as the unlocked paths of an earlier version.

A customer-row lock serializes cooperating application writers. Direct SQL that bypasses this protocol can still violate aggregate rules. There is no table-wide immutable audit trail of who collected each payment because those rows do not contain a collector user ID.

8. Accounts, passwords and session identity

`UserDao.save` and `resetPassword` lock user records in user_id order. They revalidate that the acting user is an active owner, reject duplicate usernames/normalized names, protect the current owner from self-deactivation/demotion, and retain at least one active owner. Deterministic locking reduces competing-owner races and avoids relying only on stale enabled UI controls.

Profile editing deliberately omits password changes. Password reset is independent, preventing an innocent name edit from replacing a credential. The panel's dirty-state choices Save/Discard/Cancel protect unfinished work. After saving the signed-in user's details, `Session.refresh` updates only that same still-active session identity and notifies UI listeners on the EDT.

PasswordUtil uses a random 16-byte salt, 65,536 PBKDF2-HMAC-SHA256 iterations and a 256-bit derived key. The stored format includes algorithm, iteration count, salt and hash. This describes the current implementation, not a guarantee that its work factor meets every deployment's contemporary security requirements.

There is no universal per-operation session revocation service. Account administration rechecks its actor in the DAO, while other business permissions rely on desktop/session controls. Direct database access remains a separate trust boundary.

9. Background work and navigation

Swing components must be updated on the event dispatch thread. UiTask/SwingWorker execute database work in the background and marshal completion back to that thread. Busy flags disable relevant actions, and screen contracts expose pending operations. Navigation, logout and exit guards prevent a user abandoning an in-progress write as though it never happened.

Search/report generation can finish out of order. The UI uses current request state to avoid showing an old response as the newest selection. Products and Inventory debounce search to avoid querying on every keystroke and bind literal search text. Record IDs, rather than row positions alone, preserve selection across refreshed/filtered tables.

This is explicit application coordination rather than a reactive framework. A new screen must implement the same refresh/busy/leave conventions or it can reintroduce duplicate writes and stale UI state.

10. Delivery, reports and printing

DeliveryRules allows dispatch only from Pending with a rider, and rejects changing a delivered order's rider. Completion only rejects an already Delivered state, so Pending → Delivered is also possible without dispatch. Conditional status updates compare the expected old status to detect stale screens. There is no payment side effect.

ReportDao performs sales queries; Ranking accumulates totals per customer in a HashMap and sorts them descending. For n rows and c customers, grouping is expected O(n), sorting O(c log c), with the loaded rows also consuming memory. SQL aggregation/pagination would be a scaling option, not an existing architecture change.

PrintDocument provides a captured document model and shared rendering used by preview and Printable. This avoids maintaining two layouts that drift apart. Receipt documents are 80 mm continuous rolls with fitted length and wrapped text. Report documents use A4 pagination and orientation. The preview toolbar exposes only controls meaningful for that document type. Zoom changes viewing scale rather than recording a new sale.

The printed snapshot is stable even when a screen's underlying data later refreshes. Reloading a past order still joins present-day descriptive records, however; exact historical wording would require additional stored snapshots.

11. Extension map and limitations

Change	Trace together
Add tax/discount/delivery fee	Order UI, rules, stored snapshots, schema migration, reports and receipt totals/ rounding.
Strict delivery progression	DeliveryRules plus transactional DAO status/rider checks and concurrency tests.
Cancel/refund an order	Explicit reversal records for stock, payments and loan movements; authorization and double-reversal protection.
Invoice-based collections	Payment allocation schema and accounting rules, UI selection and report semantics.
Stronger centralized authorization	A trusted service/API boundary; migrate validation out of untrusted desktop clients.
Historical receipt wording	Order/customer/product snapshot columns, old-data policy and receipt lookup.
Larger deployments	Connection management, indexed query plans, pagination, SQL ranking and concurrency/load tests.
New screen	MainFrame registration, refresh, pending-operation and dirty-form contracts, role visibility and scaling checks.

Read source in this order: Main → MainFrame/LoginPanel → Database → one simple model/DAO/ panel → Product/OrderRules/OrderDao → LedgerDao/UserDao → PrintDocument and ReportsPanel → the corresponding tests. A feature is complete only when its input, persisted meaning, calculations, reports and printed representation agree.

Verification and import provenance

The authoritative input is the uploaded `Aquatrack3-gpt-claude_1.zip`, corresponding to the user's explicitly selected NetBeans folder. The repository branch starts after the previous wrong-project import was reverted. No old AquaTrack project is restored.

Import decisions

Java source, tests, scripts, libraries and shared NetBeans metadata are byte-identical to the upload. The project README is replaced with a link to the new learning guide. Database backups, the data-bearing portable SQL dump, old JAR/ZIP backups, compiled build/dist folders and private IDE settings are excluded. The clean schema is derived only from CREATE/ALTER statements; auto-increment history is removed. The migration is extracted from the upload's appended SQL. A schema-only pre-migration variant and guarded first-owner script complete the supplied helpers' missing inputs.

The supplied usability protocol and blank observations CSV are retained under the project’s docs/ folder as historical reference material. Their mentions of survey files, private backups and pre-created local databases do not imply those resources are included or available here. No participant results are claimed.

The uploaded README refers to previous installations, prepared demo databases, 1,210 checks and a prior packaged JAR hash. Those are historical claims, not results reproduced by this import. They are not advertised as current verification.

Current checks

Compiled all application and test sources using JDK 26.0.2 with `--release 25`. Ran exactly the thirteen database-free programs listed by `scripts/build.ps1` on that runtime in headless Linux. Twelve passed; one failed. No Java source/test changes were made to hide the failure.

Program	Result
`aquatrack.ui.FixedSplitLayoutTest`	FAIL — expected sidebar 400, source specifies 420
`aquatrack.ui.DashboardLayoutTest`	PASS — 94 checks
`aquatrack.ui.LoginAndThemeTest`	PASS — 42 checks
`aquatrack.ui.WorkspaceViewportTest`	PASS — 34 checks
`aquatrack.ui.ProductInventoryFlowTest`	PASS — 29 checks
`aquatrack.ui.OrderPricingFlowTest`	PASS — 21 checks
`aquatrack.dao.OrderTransactionTest`	PASS — 28 checks

Program	Result
`aquatrack.dao.LedgerAccountsTransactionTest`	PASS — 123 checks
`aquatrack.ui.LedgerAccountsUiTest`	PASS — 43 checks
`aquatrack.util.PrintDocumentTest`	PASS — 536 checks
`aquatrack.ui.PrintPreviewFormatTest`	PASS — 11 checks
`aquatrack.ui.ReceiptPreviewTest`	PASS — 14 checks
`aquatrack.ui.ReportPrintSnapshotTest`	PASS — 20 checks

`OrderPanel.java` configures `UiKit.splitFixedLeft(details, items, 420)`. `FixedSplitLayoutTest` expects 400 for that screen and fails with “OrderPanel hidden before first visit: expected 400, got 420”. Decide the intended width before changing implementation or assertion. The PowerShell `-Test` path aborts on the first failed test and therefore does not package afterward; omit `-Test` for a build-only package. The other twelve programs were run independently here to expose their results.

Fontconfig also reported non-writable cache directories; this did not stop the passing programs. The source/test width mismatch is explicit and is not attributed to fonts.

Not exercised

No live MySQL/MariaDB server integration, Windows PowerShell execution, NetBeans/Ant build, visible desktop session, physical thermal printer, or participant study was run. `MySqlIntegrationTest` and `ScreenVisualSmokeTest` require an explicitly prepared isolated database and were not run. New setup SQL was reviewed structurally, not executed against a live server. Deployment should validate that SQL and the target printer/monitor before live use.

The repository includes deterministic SVG generation and a PDF exporter. Local link targets, SVG XML, PDF text/page bounds and rendered figures are checked during documentation preparation. PDF layout depends on installed fonts and renderer versions; byte-identical PDF output across operating systems is not promised.